Recurrent Energy Transformer
Energy Minimization with Fourier Features

Kenju Tomita · kt7432@g.rit.edu
Rochester Institute of Technology
CSCI 635: Introduction to Machine Learning
Individual Project Proposal · October 9, 2026

Abstract

I propose the **Recurrent Energy Transformer (RET)**, a complete language model that refines its internal states and predictions by minimizing learned energy. Every recurrent energy block will carry a state across tokens and refine it during inference. Fourier features will represent recent history and may support faster refinement. RET will combine my prior attention, energy, diffusion, forecasting, and specialist mechanisms. I will implement and train it from scratch, then compare it with attention and simpler RET versions. Experiments will measure prediction, recall, learning efficiency, and inference cost. Better prediction and faster adaptation are hypotheses to test.

1. Introduction and Research Objectives

Language models predict what comes next from observed text. Attention helps a Transformer connect relevant parts of that text [1]. Energy-based methods learn a score for how well a proposed state or prediction fits its context [2,3]. RET will use this score to guide revisions throughout its energy blocks and at its output. Energy here means compatibility, not physical energy.

An internal or **latent state** is the model's hidden representation of the input. RET will update these states through energy minimization. Training will learn the energy functions. At inference, trained weights will stay fixed while states and candidate predictions change. This is **inference-time adaptation**, rather than additional weight training.

The research question is: **Can recurrent energy minimization, supported by attention and Fourier-history features, improve prediction and recall at a practical cost?** My contribution will be the combined architecture and evidence about its components; established methods will be credited [2,3,9]. Efficient adaptation could benefit language-processing systems with limited computing resources, but RET's efficiency remains unproven.

Prior results, completed before this project. A three-seed CPU study of approximately 145,000-parameter models reached **2.5121 test bits per byte** for my energy/local-attention hybrid, versus **2.5782** for full attention and **2.5770** for local attention. Lower is better. Models used 128-byte context and equal 90-second training-update budgets, excluding setup and evaluation [5]. A historical 10.6-million-parameter study instead favored full attention: **1.6633 versus 1.7240 validation bits per byte** for pure energy after 40.96 million training bytes [6].

Combining mechanisms has not consistently helped. A three-seed, 24-update pilot reached **2.4265 test bits per byte**, versus **2.4247** for ordinary continuation [7]. A small synthetic decision study reached **76.95% accuracy** with an energy refiner, versus **72.66%** with its combined variant [8]. These preliminary results motivate careful comparisons; they do not establish full-RET performance.

2. Proposed Model and Learning Process

Complete model. I will independently implement RET in PyTorch, including input embeddings, positional information, attention, energy blocks, feedforward layers, and the output decoder. The initial backbone will repeat two recurrent energy blocks followed by one causal local-attention block. Attention will provide nearby context; every energy block will maintain and refine its own internal state.

Energy minimization in every energy block. On receiving a new token, a block will initialize its state from the previous refined state and the current input. A learned energy function will score that state using available context and Fourier-history features. The block will then revise the state through a small number of gradient steps. The same energy parameters will be reused across steps, and the refined state will pass to subsequent blocks and carry forward to the next token.

For one block, let $s^{(r)}$ be its state at refinement step r, h_t its observed context, and f_t its Fourier-history features. The update is:

$$s^{(r+1)} = s^{(r)} - \eta \nabla_{s^{(r)}} E_\theta(h_t, f_t, s^{(r)})$$

In plain language: **score the current state, use the gradient to suggest a change that reduces energy, and repeat.** Here, η controls the update size. This gives RET recurrence both across tokens and within each token's computation. It does not guarantee a global energy minimum or a correct prediction.

Fourier features and faster adaptation. Each energy block will transform a bounded history of its past states along the time axis into sine/cosine coordinates. These features describe slower and faster variations in that history and will condition the block's initialization and energy function. Only observed history will be used. Fourier transforms reorganize information; they do not supply future answers or guarantee faster learning. Inspired by prior Fourier token-mixing work [9], I will test whether this representation reduces the steps or time needed for useful refinement.

Combining my previous mechanisms. Feedforward and recurrent refiners, diffusion-generated candidates, forecasts, and communicating specialist blocks will propose starting states or predictions. A critic/router will combine proposals and retain an option to use the backbone's prediction. A Transformer energy network will also refine the final candidate prediction before decoding next-byte probabilities. Predicted futures will remain separate from observed-input state. Energy minimization will therefore participate throughout the model, including its output.

Learning the energy functions. Training will start from random weights and proceed in stages, followed by joint training. The main objective will be next-byte cross-entropy: rewarding correct predictions and penalizing incorrect ones. Gradients through two steps per energy block and two output-refinement steps will train energy functions to support better predictions. Forecast targets will come from a frozen snapshot of the course-trained backbone; diffusion will learn to denoise candidates. Auxiliary settings will be selected using validation data. At inference, gradients will update states and predictions only; weights will stay fixed.

Data and main comparisons. I will use enwik8 [4], a 256-byte vocabulary, fixed 90M/5M/5M-byte training/validation/test splits, and 128-byte training context. Four model families will compare full attention, the attention/energy backbone, that backbone with output energy refinement, and full RET. Approximately 250,000 parameters and three random seeds per family give 12 main fits. Sizes will match within 0.5% where feasible; actual counts will be reported. Training data, batches, the AdamW optimizer, and tuning allowances will be paired.

3. Evaluation, Feasibility, and Timeline

Research hypotheses. H1: Full RET improves prediction and recall compared with attention and simpler RET versions. H2: Fourier-history features reduce the refinement steps or inference time needed for a given prediction quality. H3: Removing state refinement or supporting mechanisms identifies which components help. These claims remain to be tested.

What will count as improvement? Text prediction will use bits per byte, where lower is better. Recall accuracy will be tested with distractors and different delays. The Brier score will compare confidence with outcomes. Learning efficiency will measure held-out prediction quality against training time; inference efficiency will measure quality against latency, gradient calls, and peak memory. Lower energy alone will not count as success.

Testing each mechanism. I will compare Fourier history with raw history and a similarly sized learned history transform. Ablations will remove per-block refinement, output refinement, diffusion, forecasts, or specialist communication individually. Inference will use 1, 2, 4, and 8 steps, varying state and output refinement separately. Costs will include Fourier transforms and all proposals; routing will not be assumed to save time.

Comparisons will match model size and data; a smaller follow-up will match total training time. Validation will select settings before final testing. Checks will cover past-only inputs, state resets, consistent serial/chunked processing, and refinement gradients. Unstable runs will be recorded. Fixed test windows and reset rules will support comparison; recall will not establish general reasoning.

Feasibility and individual work plan. I will build per-block minimization first, then add Fourier features, output refinement, and remaining modules. The planned 12 GPU hours allocate two to verification/profiling, eight to training, and two to evaluation. Profiling will set sizes with a 20% timing margin. A one-million-parameter model is optional; reduced scale will preserve repeated runs. I will implement, experiment, analyze, and write, with peer proofreading. Course code and weights will be new.

Dates	Milestone and output
October 9	Submit the proposal.
October 10–16	Build the initial backbone and present the proposal.
October 17–November 1	Implement per-block energy minimization, Fourier features, and remaining modules; verify training.
November 2–15	Train the three-seed comparisons; run recall and component tests.
November 16–29	Measure learning efficiency and refinement costs; finish validation selection and final testing.
November 30–December 8	Analyze results, prepare figures, obtain peer feedback, and finalize the report.
December 9	Submit the final report and code/artifacts.
December 10–13	Prepare and deliver the final presentation.

Expected contribution. Deliverables will be RET, comparison models, reproducible experiments, and evidence about energy-based learning and inference-time adaptation. Results will identify useful components and their costs.

References

1. Vaswani, A., et al. “Attention Is All You Need.” *NeurIPS*, 2017. Conference proceedings.

2. Gladstone, A., et al. “Energy-Based Transformers Are Scalable Learners and Thinkers.” *ICLR*, 2026. Conference proceedings.

3. Hoover, B., et al. “Energy Transformer.” *NeurIPS*, 2023. Conference proceedings.

4. Mahoney, M. “About the Test Data,” Large Text Compression Benchmark. Dataset source and checksums.

5. EnergyTransformer project. “Compute-Controlled Confirmation.” Prior local report, September 2026. Local report (local project report).

6. EnergyTransformer project. “Reconciled Evidence and Repair Results.” Prior historical-results audit, September 2026. Local report (local project report).

7. EnergyTransformer project. “Native Real-Text Pilot: Negative Result.” Prior experimental-stack study, September 2026. Local report (local project report).

8. EnergyTransformer project. “Calibration and Reproducibility Follow-up.” Prior synthetic decision-adapter study, September 2026. Local report (local project report).

9. Lee-Thorp, J., et al. “FNet: Mixing Tokens with Fourier Transforms.” *NAACL*, 2022. Conference proceedings.